

DATABASE MANAGEMENT SYSTEM



CAPSTONE PROJECT

G L NARASAMBA
VANGUR



ADITYA
UNIVERSITY

BANK ACCOUNT MANAGEMENT SYSTEM

REVIEW - 1

REVIEW - 2

ADITYA UNIVERSITY
LEARN BEYOND LIMITS



ADITYA
UNIVERSITY

— LEARN BEYOND LIMITS —

PROJECT TITLE:

BANK ACCOUNT MANAGEMENT SYSTEM



TEAM MEMBERS



25B11AI201

CHIKKALA ASHWIN



25B11AI870

PADIGAPATI VENKATA KARTHIK REDDY



25B11AI999

RACHAPOTHU TULASI MANI LAKSHMI SRI



25B11AI376

GUGULOTHU SAIDA



ADITYA
UNIVERSITY

— LEARN BEYOND LIMITS —





Problem Statement

- Banks handle thousands of **customer accounts**, **deposits**, **withdrawals**, and **transactions** every day.
- Managing this information manually is **difficult**, **time-consuming**, and prone to **errors**.
- A database system is required to securely store **customer details**, **account information**, **transaction history**, **employee records**, and **branch details**.
- The Bank Account Management System aims to automate banking operations using a relational database, ensuring **data accuracy**, **security**, and **efficient transaction management**.



ADITYA
UNIVERSITY

3. OBJECTIVES



Maintain **customer information**.



Store bank **account details**.



Record **deposits** and **withdrawals**.



Manage **transaction history**.



Manage **branch information**.



Manage **employee information**.



Generate reports using **SQL queries**.



**BANK ACCOUNT
MANAGEMENT SYSTEM**



A secure, efficient and reliable system
to manage **banking operations** effectively.

A project for
university course



5. ENTITIES

The system will maintain and manage the following core entities to ensure smooth **banking operations**.

ENTITY	DESCRIPTION
 1. CUSTOMER	Stores customer details such as name, contact information, address, etc.
 2. ACCOUNT	Stores account information including account number, type, balance, and customer reference.
 3. TRANSACTION	Stores deposits, withdrawals, and other financial transactions with date, type, amount, and account reference.
 4. BRANCH	Stores branch information such as branch name, code, address, and contact details.
 5. EMPLOYEE	Stores employee details including name, role, contact information, salary, and branch assignment.
 6. LOAN	Stores loan information including loan type, amount, interest rate, tenure, EMI, status, and account reference.



4. SCOPE

The system will provide the following features to manage **banking operations** effectively.



1. CUSTOMER REGISTRATION

The ability to add new customers to the system, gathering their personal details.



2. ACCOUNT CREATION

The feature that lets bank employees open new accounts (like checking or savings) for registered customers.



3. DEPOSIT MONEY

The function used to add funds into a customer's bank account.



4. WITHDRAW MONEY

The process that allows customers to take money out of their accounts.



5. VIEW ACCOUNT DETAILS

This feature lets users check basic information about an account, such as its current balance, type, and owner.



6. VIEW TRANSACTION HISTORY

A record of all financial activity in an account, showing past deposits, withdrawals, and other actions over time.



7. EMPLOYEE MANAGEMENT

Tools for managing bank staff, including hiring, creating profiles, assigning roles, and tracking their information.



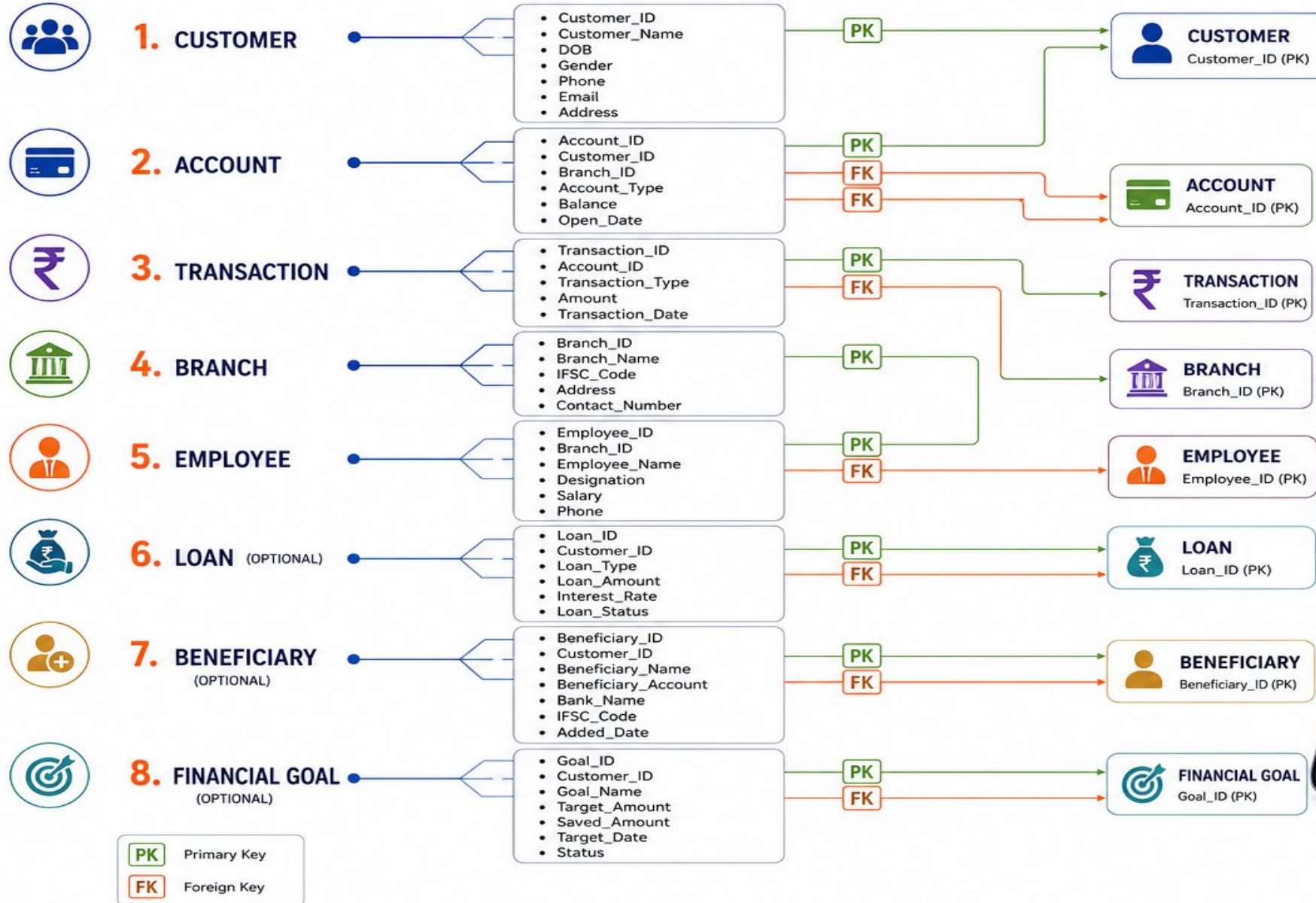
8. BRANCH MANAGEMENT

Features to oversee different physical bank locations, managing their information and data separately.



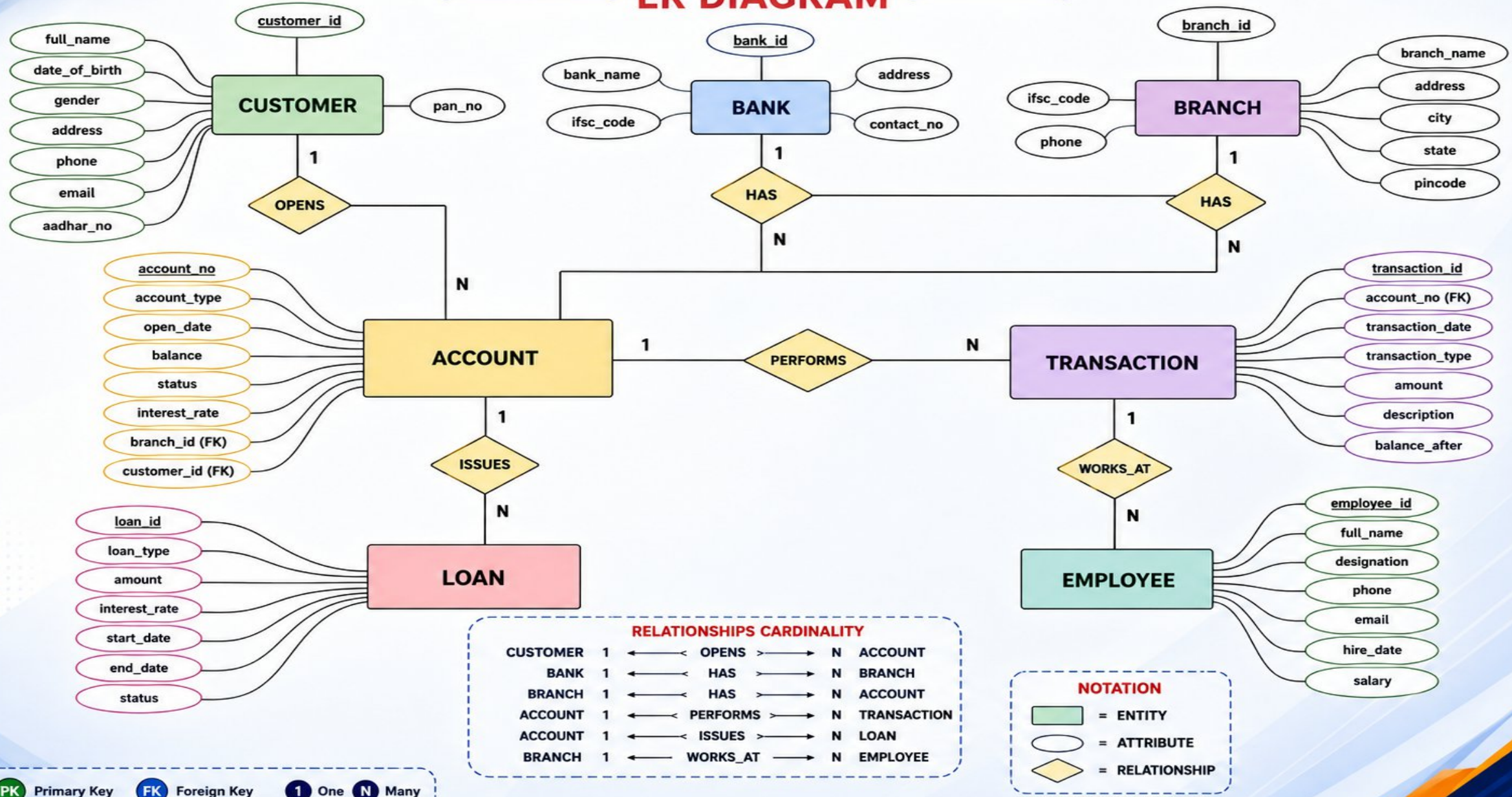
6. ATTRIBUTES

The detailed data fields and relationships for the banking entities, ensuring data integrity for smooth **banking operations**.



BANK ACCOUNT MANAGEMENT SYSTEM

ER DIAGRAM





9. PRIMARY KEYS

A **primary key** is a field (or combination of fields) that **uniquely identifies** each record in a table. It **cannot contain NULL values** and must be **unique**.

TABLE		PRIMARY KEY
	Customer	Customer_ID
	Account	Account_No
	Transaction	Transaction_ID
	Branch	Branch_ID
	Employee	Employee_ID
	Loan	Loan_ID



10. FOREIGN KEYS

A **foreign key** is a field in one table that **refers to the primary key of another table**. It is used to create **relationships between tables**.

TABLE & COLUMN			REFERENCES
	Account.Customer_ID	→	Customer.Customer_ID
	Account.Branch_ID	→	Branch.Branch_ID
	Transaction.Account_No	→	Account.Account_No
	Employee.Branch_ID	→	Branch.Branch_ID
	Loan.Customer_ID	→	Customer.Customer_ID

GITHUB REPOSITORY

 **Bank-Management-System**

A Bank Management System project for managing customers, accounts, transactions, loans, branches, and employees using a relational database.

REPOSITORY STRUCTURE

Bank-Management-System/

README.md

LICENSE

.gitignore

docs/

Project_Report.pdf

ER_Diagram.png

Database_Schema.png

Presentation.pptx

database/

bank_management.sql

tables.sql

insert_data.sql

queries.sql

src/

customer/

account/

transaction/

loan/

employee/

branch/

images/

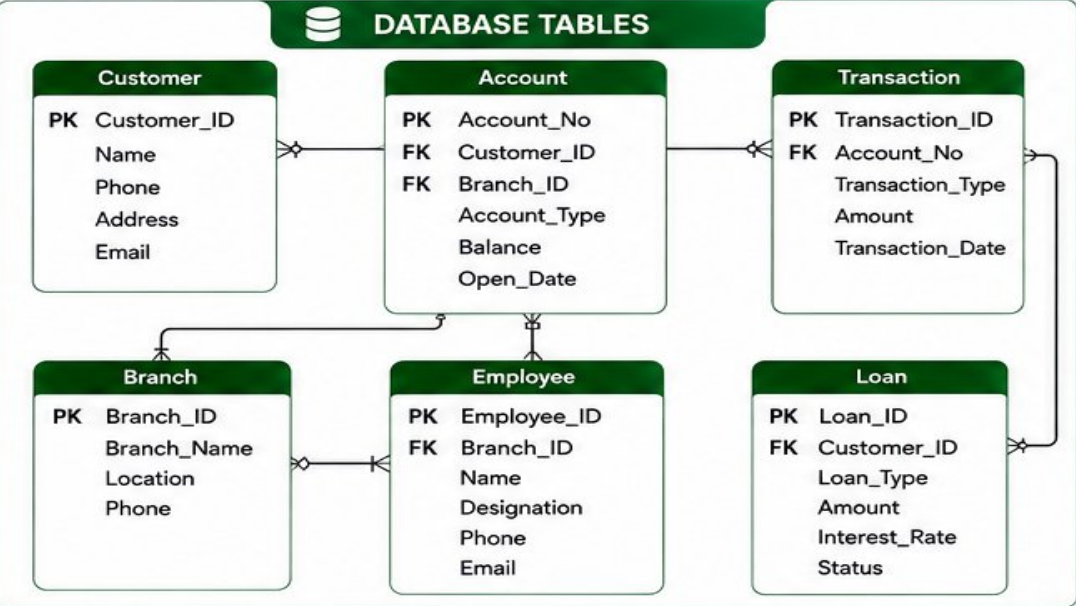
homepage.png

er_diagram.png

database_schema.png

tests/

test_cases.md



PRIMARY KEYS

Table	Primary Key
Customer	Customer_ID
Account	Account_No
Transaction	Transaction_ID
Branch	Branch_ID
Employee	Employee_ID
Loan	Loan_ID

FOREIGN KEYS

References	
Account.Customer_ID	→ Customer.Customer_ID
Account.Branch_ID	→ Branch.Branch_ID
Transaction.Account_No	→ Account.Account_No
Employee.Branch_ID	→ Branch.Branch_ID
Loan.Customer_ID	→ Customer.Customer_ID

- MODULES**
- Customer Module:** Add, update, delete, and search customer details.
 - Account Module:** Create and manage bank accounts.
 - Transaction Module:** Handle deposits, withdrawals, and transaction history.
 - Loan Module:** Process loan applications and maintain loan records.
 - Employee & Branch Module:** Manage employee and branch information.
 - Testing & Documentation:** Verify system functionality and prepare documents.

TEAM ROLES & RESPONSIBILITIES

MEMBER	ROLL NUMBER	RESPONSIBILITY
Member 1	25B11AI201	GitHub Repository Presentation
Member 2	25B11AI870	SQL Table Design, SQL Queries
Member 3	25B11AI999	Documentation
Member 4	25B11AI376	ER Diagram, Relational Schema

GIT COMMANDS (EXAMPLE)

```
git init # Initialize repository
git add . # Add all files
git commit -m "Initial project structure"
git commit -m "Added database schema and SQL scripts"
git commit -m "Implemented customer and account modules"
git commit -m "Added transaction and loan modules"
git commit -m "Updated documentation and project report"
git push origin main # Push to GitHub
```

ABOUT THE PROJECT

This project is a Bank Management System that helps manage customers, accounts, transactions, loans, branches, and employees efficiently using a relational database.

MySQL

SQL

Documentation

Relational Schema



8

Database Creation & Constraints

Bank Account Management System



Database Tables

Customer

Column Name	Data Type	Constraints
Customer_ID	INT	PK
Name	VARCHAR(50)	NOT NULL
Phone	VARCHAR(10)	NOT NULL
Email	VARCHAR(50)	UNIQUE
Address	VARCHAR(100)	NULL

Account

Column Name	Data Type	Constraints
Account_No	INT	PK
Customer_ID	INT	FK
Account_Type	VARCHAR(20)	NOT NULL
Balance	DECIMAL(10,2)	DEFAULT 0
Branch_ID	INT	FK

Transaction_Audit

Column Name	Data Type	Constraints
Audit_ID	INT	PK
Admin_ID	INT	FK
Action	VARCHAR(50)	NOT NULL
Timestamp	TIMESTAMP	NULL
Notes	VARCHAR(200)	NULL

Admin

Column Name	Data Type	Constraints
Admin_ID	INT	PK
Emp_ID	INT	FK
Role	VARCHAR(30)	NOT NULL

Branch

Column Name	Data Type	Constraints
Branch_ID	INT	PK
Branch_Name	VARCHAR(50)	NOT NULL
IFSC_Code	VARCHAR(11)	UNIQUE
Location	VARCHAR(100)	NULL

Transaction

Column Name	Data Type	Constraints
Transaction_ID	INT	PK
Account_No	INT	FK
Transaction_Type	VARCHAR(20)	NOT NULL
Amount	DECIMAL(10,2)	NOT NULL
Transaction_Date	DATE	NOT NULL

Employee

Column Name	Data Type	Constraints
Emp_ID	INT	PK
First_Name	VARCHAR(50)	NOT NULL
Last_Name	VARCHAR(50)	NOT NULL
Position	VARCHAR(50)	NULL
Branch_ID	INT	FK
Join_Date	DATE	NULL

User_Logins

Column Name	Data Type	Constraints
Login_ID	INT	PK
Customer_ID	INT	FK
Username	VARCHAR(50)	UNIQUE
Password_Hash	VARCHAR(100)	NOT NULL
Last_Login	TIMESTAMP	NULL
Security_QA	VARCHAR(200)	NULL



Constraints Used



Primary Key (PK)

Ensures each record is unique.



Foreign Key (FK)

Maintains relationships between tables.



NOT NULL

Prevents null values in mandatory fields.



UNIQUE

Ensures no duplicate values in selected fields.



CHECK

Validates data based on conditions (e.g., balance >= 0).



DEFAULT

Sets a default value when none is provided.



This structure ensures data integrity, maintains relationships, and provides a secure and efficient bank account management system.



9 – 1 Project Steps

9 Insert sample data



Customer_ID	Name	Phone	Email
C001	Rahul	9876543210	rahul@example.com
C002	Sneha	9123456780	sneha@example.com
C003	Akash	9988776655	akash@example.com

Insert sample records into the database tables for testing and verification.

1 DDL, DML & DQL operations



- DDL (CREATE, ALTER, DROP)
- DML (INSERT, UPDATE, DELETE)
- DQL (SELECT)

Perform database operations using DDL, DML and DQL commands.

2 CRUD operations & SQL queries



Implement and test CRUD operations and write required SQL queries.

3 Data integrity & validation



- ✓ Primary Key Constraints
- ✓ Foreign Key Constraints
- ✓ NOT NULL
- ✓ UNIQUE
- ✓ CHECK Constraints

Ensure data integrity and validate using constraints and rules.

4 Project documentation



- Introduction
- ER Diagram
- Schema Design
- SQL Queries
- Screenshots / Output
- Conclusion

Prepare and maintain complete project documentation.

5 Demonstrate & present database



- ✓ Run SQL queries
- ✓ Show outputs
- ✓ Explain relationships
- ✓ Highlight key features

Demonstrate the working database with proper explanation.

6 Conclusion



- Project achieved the desired objectives.
- Database is secure, consistent and efficient.
- Can be extended with additional features in the future.



1. DDL, DML & DQL Operations



DDL (CREATE, ALTER, DROP)

DDL (Data Definition Language) is used to define and modify the structure of the database.

```
CREATE TABLE Customer (  
    Customer_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(100)  
);
```

```
ALTER TABLE Customer  
    ADD Phone VARCHAR(15);
```

```
DROP TABLE Branch;
```



Creates, changes or removes the database structure.



DML (INSERT, UPDATE, DELETE)

DML (Data Manipulation Language) is used to insert, update or delete data in the database.

```
INSERT INTO Customer (Customer_ID, Name,  
    Email) VALUES (1, 'Rahul', 'rahul@example.com');
```

```
UPDATE Customer  
    SET Email = 'sneha@example.com'  
    WHERE Customer_ID = 2;
```

```
DELETE FROM Customer  
    WHERE Customer_ID = 3;
```



Modifies the data inside the tables.



DQL (SELECT)

DQL (Data Query Language) is used to retrieve data from the database.

```
SELECT Customer_ID, Name, Email  
FROM Customer  
WHERE Email LIKE '%@example.com';
```



Customer_ID	Name	Email
1	Rahul	rahul@example.com
2	Sneha	sneha@example.com



Fetches data from one or more tables.



2. CRUD Operations & SQL Queries



CREATE

(Insert New Records)

The INSERT statement is used to add new records into a table.

```
INSERT INTO Customer
(Customer_ID, Name, Phone,
Email, Address)
VALUES
(101, 'Rahul', '9876543210',
'rahul@example.com',
'Hyderabad');
```



New record is added to the table.



READ

(Retrieve Data)

The SELECT statement is used to fetch data from a table.

```
SELECT Customer_ID, Name,
Email
FROM Customer
WHERE Branch_ID = 2;
```

Customer_ID	Name	Email
1	Rahul	rahul@example.com
2	Sneha	sneha@example.com



Specified data is retrieved from the table.



UPDATE

(Modify Existing Records)

The UPDATE statement is used to modify existing data in a table.

```
UPDATE Customer
SET Email = 'rahul123@example.com'
WHERE Customer_ID = 1;
```



Record is updated in the table.



DELETE

(Remove Records)

The DELETE statement is used to remove records from a table.

```
DELETE FROM Customer
WHERE Customer_ID = 3;
```



Record is deleted from the table.



Example: Full CRUD Flow

All four operations (Create, Read, Update, Delete) work together to manage data in a database.



Insert

Add new customer record.



Select

View customer details.



Update

Edit customer information.



Delete

Remove customer record.



These operations are essential for managing data efficiently and securely.



3. Data Integrity & Validation



What it is?

Data integrity ensures that the data in the database is accurate, consistent and reliable. Validation checks are used to enforce rules and prevent incorrect or invalid data from being entered into the system.



Goal

Maintain accurate, consistent and reliable data across all tables and transactions.



Example: SQL Constraints & Validation

```
CREATE TABLE Customer (  
  Customer_ID INT PRIMARY KEY,  
  Name VARCHAR(50) NOT NULL,  
  Phone VARCHAR(10) UNIQUE,  
  Email VARCHAR(100) UNIQUE,  
  Balance DECIMAL(10,2) CHECK (Balance >= 0)  
);  
  
-- Validation example (only valid phone numbers)  
CREATE OR REPLACE FUNCTION ValidatePhone()  
  RETURNS TRIGGER AS $$  
  BEGIN  
    IF NEW.Phone !~ '^([0-9]){10}$' THEN  
      RAISE EXCEPTION 'Invalid phone number format!';  
    END IF;  
    RETURN NEW;  
  END;  
  $$ LANGUAGE plpgsql;
```



Validation Rules



NOT NULL

Prevents null values in mandatory fields.



UNIQUE

Ensures no duplicate values (e.g., email, phone).



CHECK

Validates data based on conditions (e.g., balance >= 0).



PRIMARY KEY

Ensures each record is unique and identifiable.



FOREIGN KEY

Maintains relationship between tables.



Benefits



Accurate data



Consistent records



Reduced errors



Reliable transactions





4. Project Documentation



Complete project documentation covering all aspects of the Bank Account Management System.



Includes

- 1 Introduction**
Overview of the project and its purpose.
- 2 ER Diagram**
Entity-Relationship model of the database.
- 3 Schema Design**
Tables, attributes and relationships.
- 4 SQL Queries**
Queries used for data manipulation and retrieval.
- 5 Screenshots / Output**
Sample execution results and system output.
- 6 Conclusion**
Summary of the project and key learnings.



Final Deliverable



Well-structured and complete project report.



Prepared as per project guidelines



Clear and organized format



Includes all required sections



Ready for submission and presentation!

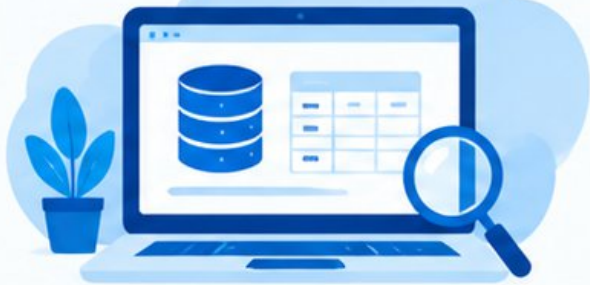


5. Demonstrate & Present Database



What it is?

The database is demonstrated using sample data and real-time queries. It shows how the data is stored, retrieved and managed in the system.



Shows the working database with proper explanation.



Sample Output (Example)

1. View Customers

Customer_ID	Name	Phone	Email	Address
C001	Rahul	9876543210	rahul@example.com	Hyderabad
C002	Sneha	9123456780	sneha@example.com	Vijayawada
C003	Akash	9988776655	akash@example.com	Guntur

2. View Account Details

Account_No	Customer_ID	Account_Type	Balance	Branch_ID
A1001	C001	Savings	50000.00	B001
A1002	C002	Current	75000.00	B002
A1003	C003	Savings	120000.00	B001

3. Transaction History (Sample)

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
T001	A1001	Deposit	10000.00	2025-08-15
T002	A1002	Withdrawal	5000.00	2025-08-16
T003	A1003	Transfer	20000.00	2025-08-17



Key Points



Run SQL Queries

Retrieve, insert, update and delete data from tables.



Display Results

Show query output in a clear and structured format.



Explain Relationships

Highlight how data is connected across tables.



Ensure Accuracy

Verify data integrity and referential rules.



A working database with real data and clear results.



6. Conclusion



Summary

The Bank Account Management System successfully demonstrates how a relational database can be used to manage customer details, accounts, transactions, employees and branches **efficiently and securely**.

It ensures data accuracy, reduces manual work, and provides a **reliable** and **organized** way to handle banking operations.



Key Takeaways



Data Security

Sensitive data is stored and accessed in a controlled manner.



Data Accuracy

Maintains consistency and reduces human errors.



Efficient Management

Automates banking operations and improves performance.



Scalability

Can handle large amounts of data and multiple branches.



Overall Impact

Provides a strong foundation for a secure, reliable and efficient banking system.



*Better Data
for a Stronger
Banking Future*



**A Real-World Database Solution
for Smarter and Safer Banking Operations.**



<https://github.com/Ash17abd/Bank-Account-Management-system>



https://docs.google.com/document/d/1_Glf-d8E7kOkstfnBjv2tXZS1D4ALghu/edit?usp=drive_link&oid=114876032609406157375&rtpof=true&sd=true



ADITYA
UNIVERSITY
— LEARN BEYOND LIMITS —

THANK YOU

Thank you for your time, attention
and valuable feedback.